VRUSHABH SAWANT SYVLSI_44 125BTVL1033

Experiment 3

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	**Semester**	III
Course	Skill Lab: Python Programming	**Course Code**	VDCOR2VS201
Class	SY VLSI	**Course In-Charge**	Mr. Nirmol Munvar
Exp Name	Dictionaries		
CO	Apply fundamental Python programming concepts such as variables, data types, operators, and input–output statements to develop basic programs for solving computational problems.		

Dictionaries

A **dictionary** is a collection used to store data in **key-value pairs**.

Dictionaries are written using curly brackets {}.

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85
}
```

Here:

"name" → key

"Rahul" → value

"age" → key

20 → value

"marks" → key

85 → value

A dictionary allows you to associate a piece of information with a name or identifier.

For example, instead of:

```
student = ["Rahul", 20, 85]
```

we can write:

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85
}
```

The second version is easier to understand because each value has a meaningful key.

Important Properties of Dictionaries

Dictionaries are:

- **Mutable** — their contents can be changed.
- Based on **key-value pairs**.
- Keys must be **unique**.

- Values can be duplicated.
- Values can have different data types.
- Keys must be of a hashable type, such as strings, integers, or tuples containing hashable values.
- Dictionaries preserve **insertion order** in modern Python.

For example:

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85
}
```

The keys are:

name
age
marks

The values are:

Rahul
20
85

Creating a Dictionary

The simplest dictionary:

```
student = {
   "name": "Rahul",
   "age": 20
}
```

You can also create an empty dictionary:

```
student = {}
```

You can then add data to it:

```
student["name"] = "Rahul"
student["age"] = 20
```

Now:

```
print(student)
```

Output:

```
{'name': 'Rahul', 'age': 20}
```

You can also use dict():

```
student = dict()
```

This also creates an empty dictionary.

Dictionary Keys and Values

Consider:

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85
}
```

Here:

Keys:

"name"
"age"
"marks"

Values:
"Rahul"
20
85
You can have different types of values:

```
data = {
    "name": "Rahul",
    "age": 20,
    "marks": 85.5,
    "passed": True
}
```

Values can even be other collections:

```
student = {
    "name": "Rahul",
    "subjects": ["Python", "Math", "English"]
}
```

Accessing Dictionary Values
You access dictionary values using their keys.

```
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 85
}

print(student["name"])
```

Output:
Rahul
Another example:

```
print(student["marks"])
```

Output:
85
The important idea is:
Lists use indexes to access elements, while dictionaries use keys.
For a list:

```
numbers = [10, 20, 30]

print(numbers[1])
```

For a dictionary:

```
student = {
    "name": "Rahul",
    "age": 20
}

print(student["age"])
```

Accessing a Key That Doesn't Exist
Suppose:

```
student = {
    "name": "Rahul",
    "age": 20
}
```

If you do:
print(student["marks"])
Python produces a KeyError because "marks" does not exist.
A safer way is to use get().
print(student.get("marks"))
Output:
None
You can also provide a default value:
print(student.get("marks", 0))
Output:
0
This is useful when you're not sure whether a key exists.

Adding a New Key-Value Pair

Dictionaries are mutable, so you can add new items.

```
student = {
   "name": "Rahul",
   "age": 20
}

student["marks"] = 85

print(student)
```

Output:
{'name': 'Rahul', 'age': 20, 'marks': 85}
You simply assign a value to a new key.

Modifying a Value

You can change the value associated with an existing key.

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85
}

student["marks"] = 95

print(student)
```

Output:
{'name': 'Rahul', 'age': 20, 'marks': 95}
The key "marks" already existed, so its value was changed from 85 to 95.
This demonstrates that dictionaries are **mutable**.

Removing Dictionary Items

There are several ways to remove items.

del

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85
}

del student["age"]
```

```
print(student)
```
Output:
```
{'name': 'Rahul', 'marks': 85}
```
del removes the key-value pair completely.

pop()

pop() removes a key and returns its value.
```
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 85
}

age = student.pop("age")

print(age)
print(student)
```
Output:
```
20
{'name': 'Rahul', 'marks': 85}
```
This is useful when you want both:

- To remove the item
- To use the value that was removed

You can also provide a default:
```
result = student.pop("address", "Not found")

print(result)
```
If "address" doesn't exist, it returns "Not found" instead of raising a KeyError.

popitem()

popitem() removes and returns the most recently inserted key-value pair.
```
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 85
}

item = student.popitem()

print(item)
print(student)
```
Output:
```
('marks', 85)
{'name': 'Rahul', 'age': 20}
```
The returned item is a tuple containing:
(key, value)

keys()

The keys() method gives you the dictionary's keys.
```
student = {
    "name": "Rahul",
    "age": 20,
```

```
    "marks": 85
}

print(student.keys())
```
You can also use it in a loop:
```
for key in student.keys():
    print(key)
```
Output:
```
name
age
marks
```
You don't need to explicitly write .keys() in a for loop:
```
for key in student:
    print(key)
```
This also iterates over the keys.

values()

The values() method gives you the dictionary's values.
```
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 85
}

print(student.values())
```
You can iterate over them:
```
for value in student.values():
    print(value)
```
Output:
```
Rahul
20
85
```
items()

The items() method gives you both keys and values.
```
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 85
}

print(student.items())
```
You will get key-value pairs.

A very common way to use it is:
```
for key, value in student.items():
    print(key, value)
```
Output:
```
name Rahul
age 20
marks 85
```
This is one of the most important dictionary patterns to learn.

Checking Whether a Key Exists

The in operator can be used with dictionaries.

```
student = {
   "name": "Rahul",
   "age": 20
}

print("name" in student)
```

Output:
True
And:

```
print("marks" in student)
```

Output:
False
When used directly with a dictionary, in checks for **keys**.
For example:

```
student = {
   "name": "Rahul",
   "age": 20
}

print("Rahul" in student)
```

Output:
False
Why?
Because "Rahul" is a value, not a key.

Checking Values

If you want to check whether a value exists, use values():

```
student = {
   "name": "Rahul",
   "age": 20
}

print("Rahul" in student.values())
```

Output:
True

len() with Dictionaries

len() tells you how many key-value pairs are present.

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85
}

print(len(student))
```

Output:
3
There are three key-value pairs.

update()

update() can be used to add or modify multiple key-value pairs.

```
student = {
```

```
    "name": "Rahul",
    "age": 20
}

student.update({
    "age": 21,
    "marks": 90
})

print(student)
```
Output:
{'name': 'Rahul', 'age': 21, 'marks': 90}
Notice:

- "age" already existed, so it was modified.
- "marks" didn't exist, so it was added.

clear()
clear() removes all items from a dictionary.
```
student = {
    "name": "Rahul",
    "age": 20
}

student.clear()

print(student)
```
Output:
{}
The dictionary still exists, but it is now empty.
Dictionary Mutability
Dictionaries are **mutable**.
That means we can modify their contents after creating them.
For example:
```
student = {
    "name": "Rahul",
    "age": 20
}

student["age"] = 21
```
The dictionary itself has been modified.
Compare this with a tuple:
```
student = ("Rahul", 20)
```
You cannot do:
```
student[1] = 21
```
because tuples are immutable.
So:
List → Mutable
Dictionary → Mutable
Set → Mutable
Tuple → Immutable
String → Immutable

Dictionary Keys Must Be Unique

A dictionary cannot have two separate values associated with the same key.

For example:

```
student = {
   "name": "Rahul",
   "name": "Amit"
}

print(student)
```

The result will be:

```
{'name': 'Amit'}
```

The second "name" replaces the first one.

So this:

"name": "Rahul"

"name": "Amit"

does not create two entries.

The key "name" occurs only once.

Dictionary Values Can Be Duplicated

While keys must be unique, values don't have to be.

This is perfectly valid:

```
students = {
   "Rahul": 90,
   "Amit": 90,
   "Priya": 85
}
```

Both Rahul and Amit have the value 90.

Dictionary Keys and Data Types

Dictionary keys need to be **hashable**.

Common valid key types include:

```
{
   "name": "Rahul",
   1: "one",
   2: "two",
   (10, 20): "coordinate"
}
```

Strings, integers, and tuples can be used as keys when they are hashable.

A list cannot be used as a key:

```
data = {
   [1, 2]: "numbers"
}
```

This produces an error because lists are mutable and therefore not hashable.

A simple beginner rule is:

Dictionary keys should generally be immutable/hashable types such as strings, numbers, and suitable tuples.

Dictionaries Can Contain Lists

Values can be lists.

For example:

```
student = {
   "name": "Rahul",
   "subjects": ["Python", "Math", "English"]
```

}
You can access the list:
print(student["subjects"])
Output:
['Python', 'Math', 'English']
You can then use list indexing:
print(student["subjects"][0])
Output:
Python
And you can modify the list because the list itself is mutable:
student["subjects"].append("Physics")

print(student)
Output:
{
  'name': 'Rahul',
  'subjects': ['Python', 'Math', 'English', 'Physics']
}
This is a useful example of **nested data structures**.
Dictionaries Can Contain Dictionaries
A dictionary can also contain another dictionary as a value.
students = {
  "student1": {
    "name": "Rahul",
    "age": 20
  },
  "student2": {
    "name": "Amit",
    "age": 21
  }
}
You can access nested values:
print(students["student1"]["name"])
Output:
Rahul
And:
print(students["student2"]["age"])
Output:
21
The general idea is:
dictionary
  ↓
key
  ↓
another dictionary
  ↓
key
  ↓
value
Nested Dictionaries and Lists

You can combine multiple structures.

```
students = {
   "student1": {
      "name": "Rahul",
      "subjects": ["Python", "Math"]
   },
   "student2": {
      "name": "Amit",
      "subjects": ["Physics", "English"]
   }
}
```

Accessing "Python":

```
print(students["student1"]["subjects"][0])
```

Output:

Python

Here Python performs the operations from left to right:

students
↓
"student1"
↓
"subjects"
↓
[0]
↓
"Python"

Looping Through a Dictionary

You can use a for loop to process dictionary data.

Loop through keys

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85
}

for key in student:
   print(key)
```

Output:

name
age
marks

Loop through values

```
for value in student.values():
   print(value)
```

Output:

Rahul
20
85

Loop through keys and values

```
for key, value in student.items():
   print(key, ":", value)
```

Output:
name : Rahul
age : 20
marks : 85
This is a very common pattern in Python.

Dictionary Methods Summary

Method	Purpose
get()	Safely get a value
keys()	Get keys
values()	Get values
items()	Get key-value pairs
update()	Add/modify multiple items
pop()	Remove a specific key
popitem()	Remove the last inserted pair
clear()	Remove everything

Some useful operations are functions rather than methods:
len(dictionary)
and operators:
key in dictionary

Dictionary vs List

This distinction is very important.
A list:
students = ["Rahul", "Amit", "Priya"]
uses indexes:
print(students[0])
Output:
Rahul
A dictionary:

```
students = {
   "first": "Rahul",
   "second": "Amit",
   "third": "Priya"
}
```

uses keys:
print(students["first"])
Output:
Rahul
Think:
LIST
position → value

DICTIONARY
key → value

Dictionary vs Set

Both dictionaries and sets use curly brackets.
A set:
numbers = {10, 20, 30}
contains values only.

A dictionary:

```
student = {
   "name": "Rahul",
   "age": 20
}
```

contains key-value pairs.
An empty {} is a dictionary:

```
x = {}
```

An empty set must be created using:

```
x = set()
```

Dictionary Unpacking
Dictionaries can be unpacked in useful ways.
For example:

```
student = {
   "name": "Rahul",
   "age": 20
}
```

You can get its keys:

```
keys = student.keys()
```

its values:

```
values = student.values()
```

and its key-value pairs:

```
items = student.items()
```

Remember that these methods return special view objects, which can be iterated over.
Practice Exercises
Dictionary Basics
Create a dictionary called student containing:
name
age
course
marks
For example:

```
student = {
   "name": "Rahul",
   "age": 20,
   "course": "Python",
   "marks": 85
}
```

Print:

- The student's name
- The student's age
- The course
- The marks

Also print:

```
type(student)
```

Adding and Modifying Data
Given:

```
student = {
   "name": "Rahul",
   "age": 20
```

}
Do the following:

- Add "course": "Python".
- Add "marks": 90.
- Change the age to 21.
- Print the final dictionary.

Expected result:
{'name': 'Rahul', 'age': 21, 'course': 'Python', 'marks': 90}

Removing Data

Given:

```
student = {
   "name": "Rahul",
   "age": 20,
   "course": "Python",
   "marks": 90
}
```

Do the following:

- Remove "age" using del.
- Remove "marks" using pop().
- Print the resulting dictionary.

Dictionary Methods Exercise

Given:

```
student = {
   "name": "Rahul",
   "age": 20,
   "course": "Python",
   "marks": 90
}
```

Use:
keys()
values()
items()
to print:

- All keys
- All values
- All key-value pairs

Then use:
len()
to find the number of key-value pairs.

get() Exercise

Given:

```
student = {
   "name": "Rahul",
   "age": 20
}
```

Try:

```
print(student.get("name"))
print(student.get("marks"))
```

Then try:

```
print(student.get("marks", 0))
```

Understand the difference between:
student["marks"]
and:
student.get("marks")

in Operator Exercise

Given:

```
student = {
   "name": "Rahul",
   "age": 20,
   "course": "Python"
}
```

Check whether the following keys exist:
"name"
"marks"
"course"
"email"
Use the in operator.

Dictionary Loop Exercise

Given:

```
student = {
   "name": "Rahul",
   "age": 20,
   "course": "Python",
   "marks": 90
}
```

Write three separate loops:

- One that prints all keys.
- One that prints all values.
- One that prints both keys and values.

The final loop should produce something similar to:
name : Rahul
age : 20
course : Python
marks : 90

Nested Dictionary Exercise

Create:

```
students = {
   "student1": {
      "name": "Rahul",
      "age": 20,
      "marks": 85
   },
   "student2": {
      "name": "Amit",
      "age": 21,
      "marks": 90
   }
}
```

Print:

- Rahul's name

- Rahul's marks
- Amit's age
- Amit's marks

For example:

```
print(students["student1"]["name"])
print(students["student1"]["marks"])
```

Dictionary with Lists Exercise

Create:

```
student = {
   "name": "Rahul",
   "subjects": ["Python", "Math", "English"]
}
```

Do the following:

- Print the student's name.
- Print the first subject.
- Print the last subject.
- Add "Physics" to the subjects list.
- Print the updated dictionary.

Input and Dictionary Exercise

Write a program that asks the user for:

Name

Age

Course

Marks

Store the information in a dictionary.

For example:

```
Enter name: Rahul
Enter age: 20
Enter course: Python
Enter marks: 85
```

The resulting dictionary should conceptually look like:

```
{
   "name": "Rahul",
   "age": 20,
   "course": "Python",
   "marks": 85
}
```

Then print each key and value.

Final Challenge

Write a program that stores information about **three students** using a nested dictionary.

Your data should look something like:

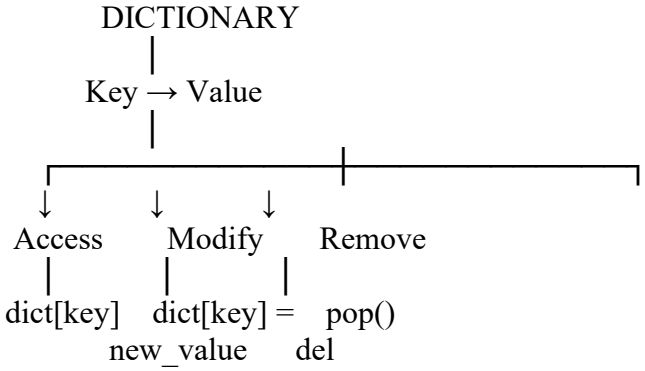
```
students = {
   "student1": {
      "name": "Rahul",
      "age": 20,
      "marks": 85
   },
   "student2": {
      "name": "Amit",
      "age": 21,
```

```
        "marks": 90
    },
    "student3": {
        "name": "Priya",
        "age": 19,
        "marks": 95
    }
}
```

Then write code to:

- Print all three students.
- Print Rahul's marks.
- Print Priya's age.
- Print each student's name and marks using a loop.
- Add a new student.
- Modify one student's marks.
- Remove one student.
- Check whether "student2" exists.
- Print the number of students using len().

Key Concepts to Master



Remember:

Dictionary → key-value pairs
Keys → unique
Values → can be duplicated
Mutable → yes
Indexing → no positional indexing
Access → using keys
in → checks keys
get() → safely retrieves values
items() → gives key-value pairs
keys() → gives keys
values() → gives values

And the overall collection picture is:

List
→ Ordered
→ Mutable
→ Index-based
→ Duplicates allowed

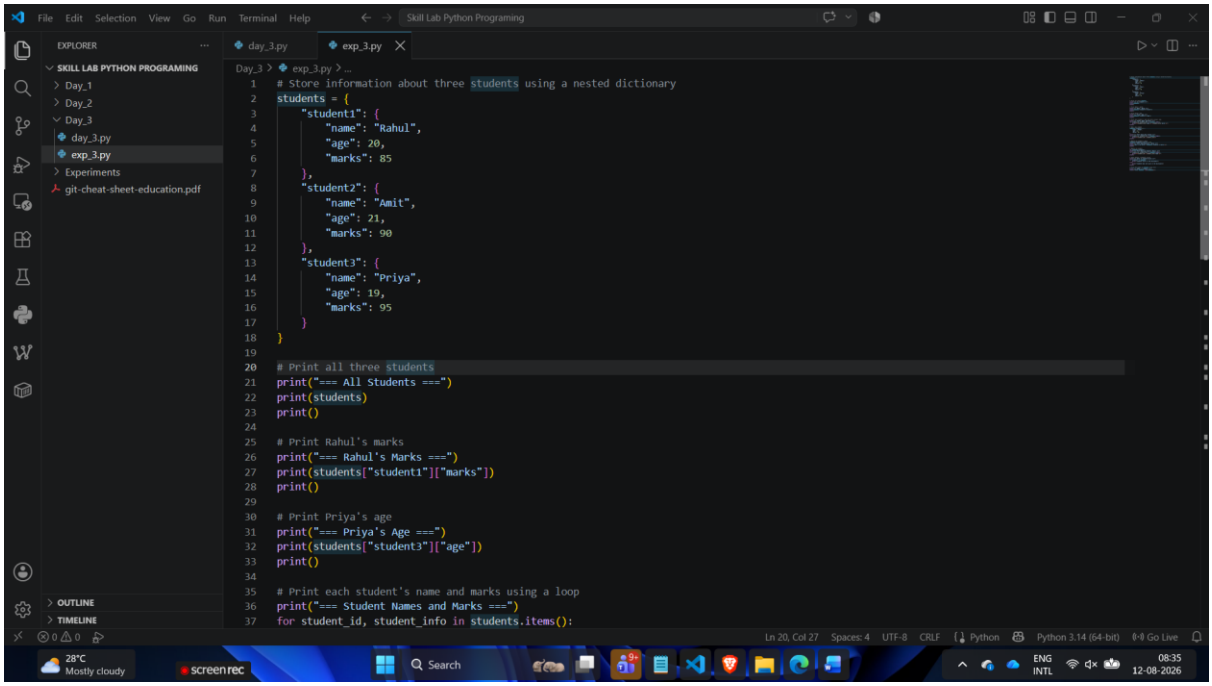
Tuple
→ Ordered

→ Immutable
→ Index-based
→ Duplicates allowed

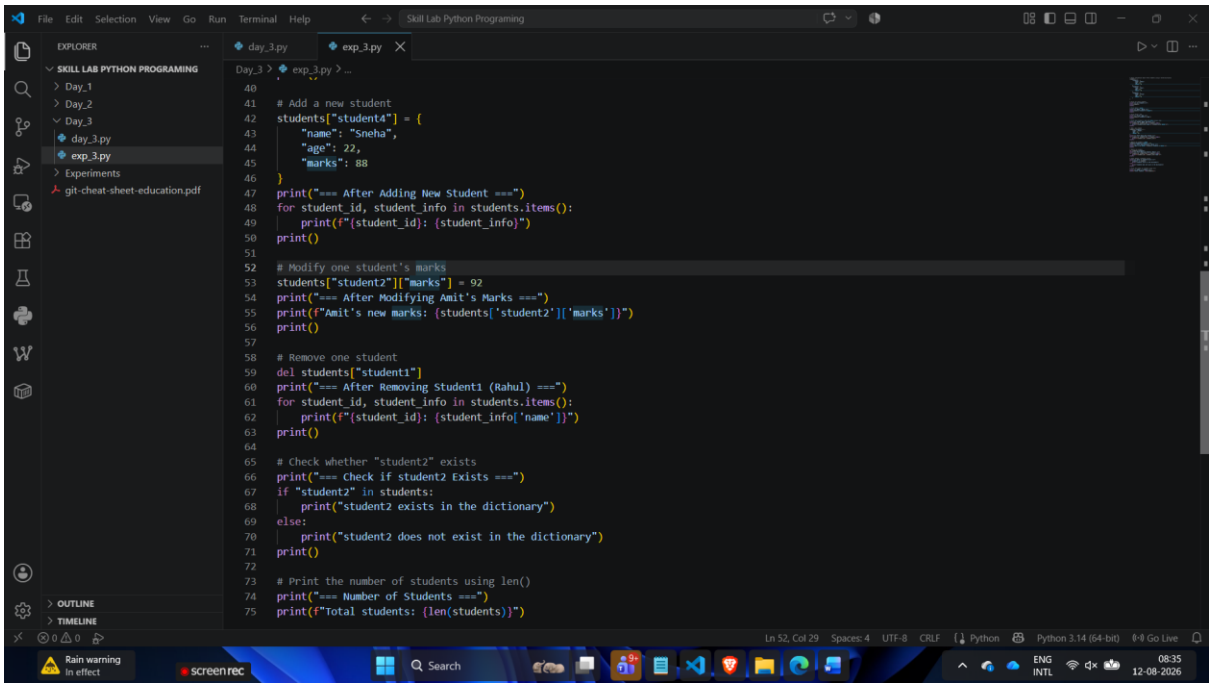
Set
→ Unique elements
→ Mutable
→ No positional indexing

String
→ Ordered characters
→ Immutable
→ Indexing and slicing

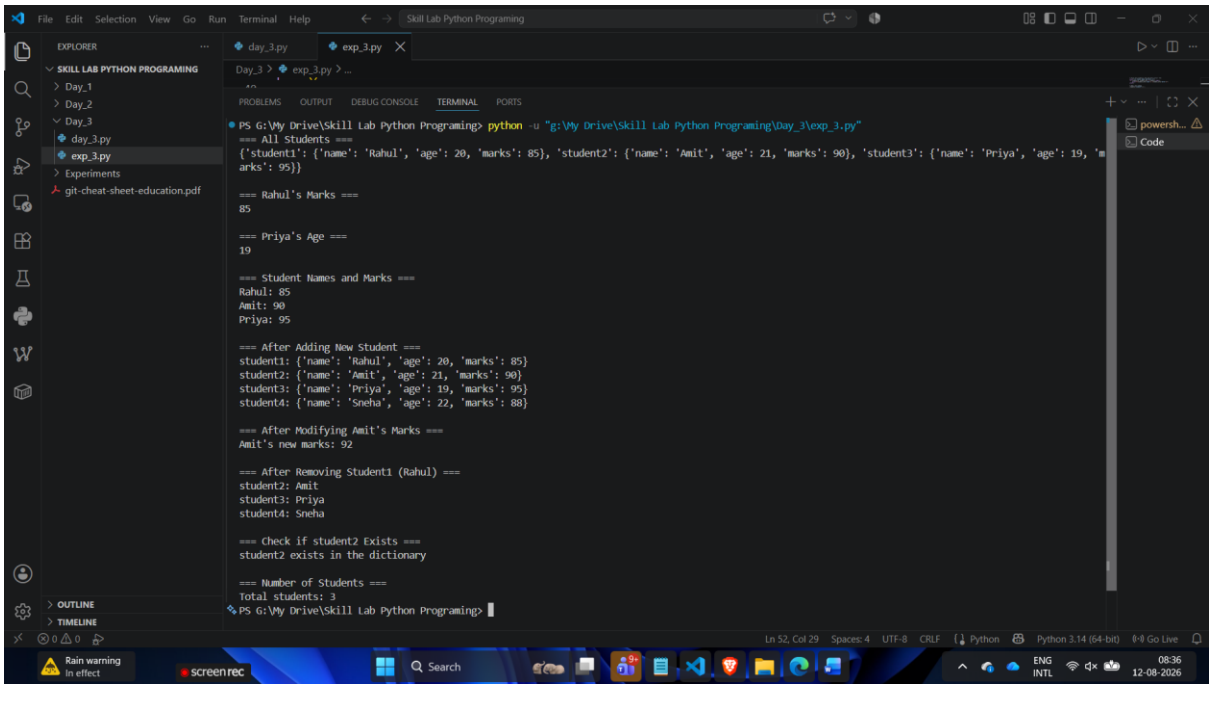
Dictionary
→ Key-value pairs
→ Mutable
→ Keys are unique
→ Accessed using keys
A particularly important concept to remember is:
A list answers "What is at position 2?" while a dictionary answers "What value belongs to this key?"



```python
# Store information about three students using a nested dictionary
students = {
    "student1": {
        "name": "Rahul",
        "age": 20,
        "marks": 85
    },
    "student2": {
        "name": "Amit",
        "age": 21,
        "marks": 90
    },
    "student3": {
        "name": "Priya",
        "age": 19,
        "marks": 95
    }
}

# Print all three students
print("=== All Students ===")
print(students)
print()

# Print Rahul's marks
print("=== Rahul's Marks ===")
print(students["student1"]["marks"])
print()

# Print Priya's age
print("=== Priya's Age ===")
print(students["student3"]["age"])
print()

# Print each student's name and marks using a loop
print("=== Student Names and Marks ===")
for student_id, student_info in students.items():
```



```python

# Add a new student
students["student4"] = {
    "name": "Sneha",
    "age": 22,
    "marks": 88
}
print("=== After Adding New Student ===")
for student_id, student_info in students.items():
    print(f"{student_id}: {student_info}")
print()

# Modify one student's marks
students["student2"]["marks"] = 92
print("=== After Modifying Amit's Marks ===")
print(f"Amit's new marks: {students['student2']['marks']}")
print()

# Remove one student
del students["student1"]
print("=== After Removing Student1 (Rahul) ===")
for student_id, student_info in students.items():
    print(f"{student_id}: {student_info['name']}")
print()

# Check whether "student2" exists
print("=== Check if student2 Exists ===")
if "student2" in students:
    print("student2 exists in the dictionary")
else:
    print("student2 does not exist in the dictionary")
print()

# Print the number of students using len()
print("=== Number of Students ===")
print(f"Total students: {len(students)}")
```



```
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_3\exp_3.py"
=== All Students ===
{'student1': {'name': 'Rahul', 'age': 20, 'marks': 85}, 'student2': {'name': 'Amit', 'age': 21, 'marks': 90}, 'student3': {'name': 'Priya', 'age': 19, 'marks': 95}}

=== Rahul's Marks ===
85

=== Priya's Age ===
19

=== Student Names and Marks ===
Rahul: 85
Amit: 90
Priya: 95

=== After Adding New Student ===
student1: {'name': 'Rahul', 'age': 20, 'marks': 85}
student2: {'name': 'Amit', 'age': 21, 'marks': 90}
student3: {'name': 'Priya', 'age': 19, 'marks': 95}
student4: {'name': 'Sneha', 'age': 22, 'marks': 88}

=== After Modifying Amit's Marks ===
Amit's new marks: 92

=== After Removing Student1 (Rahul) ===
student2: Amit
student3: Priya
student4: Sneha

=== Check if student2 Exists ===
student2 exists in the dictionary

=== Number of Students ===
Total students: 3
PS G:\My Drive\Skill Lab Python Programing>
```